製造業BPO詳細設計

スコープ: 中小製造業（金属加工/板金/樹脂成型/食品OEM等）10-300名規模
対象社数: 約46,000社（T2/T3/TI）
市場規模: 約44兆円/年（経産省 工業統計）
BPO月額: BPOコア¥250,000（①-④）+ 追加モジュール¥100,000/個（⑤-⑧）
ゲノム系統: 製造業専用（見積構造型 Type A/B/C/D で分岐）
パイロット: 金属切削 S1セグメント（民生・雑多 × TI/T3）10-50名規模
上位文書: e_02（全貌）、e_02a（セグメントモデル）
既存実装: `workers/bpo/manufacturing/pipelines/` 配下に全8パイプライン実装済み

---

0. エグゼクティブサマリー

なぜ製造業BPOか

```
中小製造業の現実：
  ・見積に1件30分–3時間。月100件超の引合い。回答遅延=受注逸失
  ・熟練見積担当の平均年齢57歳。10年後に半減（経産省）
  ・赤字受注率15–20%。原価割れに気付かず納品（日刊工業新聞）
  ・生産計画が工場長の頭の中。突発受注で計画崩壊→納期遅延
  ・品質データは取っているが分析できていない。不良の3–5%が放置
  ・在庫が多すぎるか足りないかの二択。発注忘れでライン停止
  ・手順書が5年前のまま。外国人実習生に通じない
  ・ISO審査前に品質管理責任者が1人で泣きながら準備

 → 全部AIで代行 = 年間1,000–1,500万円の価値
 → 見積AIで入って8モジュールに横展開
```

8モジュール全体像

```
┌──────────────────────────────────────────────────────────────┐
│  製造業BPO（BPOコア¥250,000 + 追加モジュール¥100,000/個）   │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ★ Phase A（Week 1-2）: 最優先                               │
│  ┌──────────────────┐  ┌──────────────────────┐            │
│  │ ① 見積AI          │  │ ② 生産計画AI          │            │
│  │ 3層エンジン×4類型  │  │ 山積み/山崩し→ガントチャート │    │
│  │ ¥10-30万/月の価値  │  │ ¥5-10万/月の価値       │            │
│  └──────────────────┘  └──────────────────────┘            │
│                                                              │
│  Phase B（Week 3）:                                          │
│  ┌──────────────────┐  ┌──────────────────────┐            │
│  │ ③ 品質管理        │  │ ④ 在庫最適化          │            │
│  │ SPC+Cp/Cpk+管理図  │  │ ABC分析+安全在庫+EOQ  │            │
│  └──────────────────┘  └──────────────────────┘            │
│                                                              │
│  Phase C（Week 4）:                                          │
│  ┌──────────────────┐  ┌──────────────────────┐            │
│  │ ⑤ SOP管理         │  │ ⑥ 設備保全            │            │
│  │ 手順書自動生成+多言語│  │ MTBF/MTTR+保全カレンダー │        │
│  └──────────────────┘  └──────────────────────┘            │
│                                                              │
│  Phase D（Week 5）:                                          │
│  ┌──────────────────┐  ┌──────────────────────┐            │
│  │ ⑦ 仕入管理        │  │ ⑧ ISO文書管理         │            │
│  │ BOM→MRP→発注書自動 │  │ 9001/14001+監査チェックリスト│    │
│  └──────────────────┘  └──────────────────────┘            │
│                                                              │
│  横断:                                                       │
│  ┌──────────────────────────────────────────────────┐       │
│  │ SaaS連携層（kintone / 生産管理SaaS CSV連携）      ||       │
│  └──────────────────────────────────────────────────┘       │
│  ┌──────────────────────────────────────────────────┐       │
│  │ ゲノムDB（チャージレート/材料単価/工程マスタ/得意先別ルール）||│
│  └──────────────────────────────────────────────────┘       │
└──────────────────────────────────────────────────────────────┘
```

#	モジュール	Tier	月額価値	Phase	実装場所
1	見積AI	★キラー	¥10-30万	A	`workers/bpo/manufacturing/pipelines/quoting_pipeline.py`
2	生産計画AI	★キラー	¥5-10万	A	`workers/bpo/manufacturing/pipelines/production_planning_pipeline.py`
3	品質管理	Tier1	¥3-5万	B	`workers/bpo/manufacturing/pipelines/quality_control_pipeline.py`
4	在庫最適化	Tier1	¥3-5万	B	`workers/bpo/manufacturing/pipelines/inventory_optimization_pipeline.py`
5	SOP管理	Tier2	¥2-3万	C	`workers/bpo/manufacturing/pipelines/sop_management_pipeline.py`
6	設備保全	Tier2	¥3-5万	C	`workers/bpo/manufacturing/pipelines/equipment_maintenance_pipeline.py`
7	仕入管理	Tier2	¥2-3万	D	`workers/bpo/manufacturing/pipelines/procurement_pipeline.py`
8	ISO文書管理	Tier2	¥2-3万	D	`workers/bpo/manufacturing/pipelines/iso_document_pipeline.py`

1. 見積AI（★キラーフィーチャー）

1.1 課題（金額換算の価値）

```
■ 直接効果①：見積工数の削減
  月100件 × 2時間/件 = 月200時間
  → AI化で月50時間に削減 = 150時間削減
  時給3,000円 × 150時間 = 月45万円 = 年540万円

■ 直接効果②：回答スピード改善による受注増
  回答リードタイム 4日 → 1日
  → 月2件の受注増 × 平均¥50万 × 粗利20% = 月20万円 = 年240万円

■ 直接効果③：赤字受注の防止
  赤字受注率 15% → 5%改善
  年間売上5億 × 赤字率10%改善 × 粗利20% = 年1,000万円

■ 合計効果：年1,780万円
  → BPO月28万円（年336万円）で ROI 5.3倍
```

1.2 業務フロー

```
【引合い受領】
  図面PDF/DXF + 仕様（材質、数量、表面処理）がメールで届く
    → ★AI Step 1: spec_reader（仕様書・図面データ取得）
    → ★AI Step 2: process_estimator（工程・工数推定）
    → ★AI Step 3: price_calculator（見積金額計算）
    → ★AI Step 4: output_validator（バリデーション）
    → 見積担当が最終確認→微修正→提出

【リピート品】
  過去見積DB自動参照 → 材料単価変動を反映 → ワンクリック再見積

【特急案件】
  設備稼働率を参照 → 受注可否判定 → 特急料率自動適用
```

1.3 パイプラインステップ定義

```
実装: workers/bpo/manufacturing/pipelines/quoting_pipeline.py
クラス: QuotingPipeline
エントリ: run_quoting_pipeline(company_id, input_data)
補助: run_quoting_engine(company_id, input_data) ← 3層エンジン版
```

Step 1: spec_reader（仕様書・図面データ取得）

項目	内容
マイクロエージェント	`micro/spec_reader`（LLM構造化抽出）
入力	直渡しJSON or `text` キー（仕様テキスト） or `file_path` キー（テキストファイル）
処理	レコード正規化、必須フィールド補完、LLMによる仕様テキストからの構造化抽出
出力	`dict`（product_name, material, quantity, processes[], material_weight_kg, material_unit_price, order_type, delivery_days）
エラー条件	テキスト空 → confidence=0.5で継続、全フィールドデフォルト値
LLM使用	テキスト/ファイル入力時のみ。tier=STANDARD、temperature=0.1

Step 2: process_estimator（工程・工数推定）

項目	内容
マイクロエージェント	`micro/process_estimator`
入力	Step 1の構造化データ
処理	工程ごとに加工費を分解（チャージレート × 工数 × 数量）
出力	`dict` （quote_items[], total_material_cost）
計算ロジック	下記「1.4 計算ロジック」参照

加工費計算式

```
# 工程別加工費
processing_cost_per_unit = hourly_rate * (estimated_hours + setup_hours)
processing_cost_total = processing_cost_per_unit * quantity

# 材料費
total_material_cost = material_weight_kg * material_unit_price * quantity
```

Step 3: price_calculator（見積金額計算）

項目	内容
マイクロエージェント	`micro/price_calculator`
入力	Step 2の工程別明細 + 材料費
処理	原価積み上げ + 利益率で最終見積金額算出
出力	`dict` （cost_subtotal, profit_rate, total_amount）
計算ロジック	`total_amount = cost_subtotal / (1 - profit_rate)`

Step 4: output_validator（バリデーション）

項目	内容
マイクロエージェント	`micro/output_validator`
入力	全ステップの処理結果
処理	金額ゼロチェック、工程データ存在チェック、受注区分妥当性チェック
出力	`QuotingPipelineResult` （見積総額、工程別明細、confidence、警告リスト）

1.4 計算ロジック

チャージレート参考値表（設備別）

設備	チャージレート相場	実装定数キー
汎用旋盤	¥3,500-5,000/h	-
CNC旋盤	¥6,000-8,000/h	`旋盤加工` : ¥8,000
マシニングセンタ（MC）	¥8,000-12,000/h	`フライス加工` : ¥9,000
5軸MC	¥10,000-15,000/h	-
研磨盤	¥5,000-7,000/h	`研削加工` : ¥10,000
ワイヤーカット	¥10,000-15,000/h	-
レーザー加工機	¥6,000-10,000/h	-
溶接	¥5,000-8,000/h	`溶接` : ¥7,000
板金加工	¥4,000-7,000/h	`板金加工` : ¥6,000
プレス加工	¥3,000-5,000/h	`プレス加工` : ¥5,000
表面処理	¥3,000-5,000/h	-
樹脂成型	¥6,000-10,000/h	`樹脂成型` : ¥8,000
デフォルト	¥7,500/h	`default` : ¥7,500

```
# 実装: quoting_pipeline.py
MACHINING_HOURLY_RATES: dict[str, int] = {
    "旋盤加工": 8_000,
    "フライス加工": 9_000,
    "研削加工": 10_000,
    "溶接": 7_000,
    "板金加工": 6_000,
    "プレス加工": 5_000,
    "樹脂成型": 8_000,
    "default": 7_500,
}
```

利益率テーブル

受注区分	利益率	適用条件	実装キー
標準	25%	通常案件	`standard`
短納期	35%	1週間以内納期。段取り費加算	`rush`
試作	40%	1-5個。段取り比率が高い	`prototype`
量産	15-20%	500個超。大量発注ディスカウント	`large_lot`

```
# 実装: quoting_pipeline.py
PROFIT_RATES: dict[str, float] = {
    "standard": 0.25,
    "rush": 0.35,
    "large_lot": 0.15,
    "prototype": 0.40,
}
```

材料単価参考値

材質	単価（円/kg）	JIS規格	用途
SS400	¥120	JIS G 3101	一般構造用圧延鋼材。最も汎用
S45C	¥150	JIS G 4051	機械構造用炭素鋼。シャフト等
SUS304	¥450	JIS G 4303	オーステナイト系ステンレス
SUS316	¥600	JIS G 4303	耐食性ステンレス。化学/食品
A5052	¥600	JIS H 4000	アルミ合金。板金加工向き
A7075	¥1,000	JIS H 4000	超々ジュラルミン。航空宇宙
C1100	¥1,000	JIS H 3100	純銅。導電部品
C3604	¥1,200	JIS H 3250	快削黄銅。精密加工品
PEEK	¥15,000	-	エンジニアリングプラスチック
POM	¥800	-	ポリアセタール。ギア等

見積の4類型（L3: 見積構造型）

型	計算式	対応業種
Type A	`(段取り時間 + サイクルタイム × 数量) / 60 × チャージレート + 材料費 + 外注費`	金属切削/樹脂成型
Type B	`切断長(mm) × mm単価 + 曲げ回数 × 曲げ単価 + 溶接長 × 溶接単価`	板金/印刷
Type C	`Σ(部品費) + 設計費 + 組立費 + 試運転費 + 輸送費`	機械メーカー/家具
Type D	`Σ(原料 × 配合比 × kg単価) × バッチ数 + 工程費 + 包装費 + 検査費`	食品OEM/化学受託

```
# Type A 見積計算例: S45Cシャフト φ30×100mm, 100個, CNC旋盤
setup_min = 45          # 段取り45分
cycle_min = 8           # サイクルタイム8分/個
quantity = 100
charge_rate = 6_000     # ¥6,000/h

machining_cost = (setup_min + cycle_min * quantity) / 60 * charge_rate
# = (45 + 800) / 60 × 6,000 = ¥84,500

material_cost = 0.83 * 150 * 1.15 * quantity  # 0.83kg × ¥150/kg × ロス15% × 100個
# = ¥14,318

surface_treatment = 300 * quantity  # 黒染め ¥300/個
# = ¥30,000

inspection = 5_000  # 検査費
overhead = (machining_cost + material_cost + surface_treatment + inspection) * 0.15
# = ¥20,073

subtotal = machining_cost + material_cost + surface_treatment + inspection + overhead
profit = subtotal * 0.25 / (1 - 0.25)  # 利益率25%
total = subtotal + profit
# ≈ ¥177,000（単価¥1,770/個）
```

1.5 データモデル

```
@dataclass
class QuotingPipelineResult:
    """見積パイプラインの最終結果"""
    product_name: str              # 製品名
    material: str                  # 材質（SS400/SUS304等）
    quantity: int                  # 数量
    order_type: str                # 受注区分（standard/rush/prototype/large_lot）
    quote_items: list[dict]        # 工程別明細
    total_material_cost: int       # 材料費合計（円）
    total_processing_cost: int     # 加工費合計（円）
    total_amount: int              # 見積総額（円）
    confidence: float              # 信頼度（0.0–1.0）
    warnings: list[str]            # 警告リスト
    steps_executed: list[str]      # 実行済みステップ
```

入力データ JSON構造

```
{
  "product_name": "ステンレス角フランジ",
  "material": "SUS304",
  "quantity": 10,
  "processes": [
    {"process_name": "旋盤加工", "estimated_hours": 2.5, "setup_hours": 0.5},
    {"process_name": "研削加工", "estimated_hours": 1.0, "setup_hours": 0.25}
  ],
  "material_weight_kg": 2.5,
  "material_unit_price": 1200,
  "order_type": "standard",
  "delivery_days": 14
}
```

1.6 マイクロエージェント命名マッピング

Step	マイクロエージェント	実装ファイル	責務
1	`mfg/spec_reader`	`quoting_pipeline.py#spec_reader`	仕様テキストからLLMで構造化抽出
2	`mfg/process_estimator`	`quoting_pipeline.py#process_estimator`	工程別の材料費・加工費分解
3	`mfg/price_calculator`	`quoting_pipeline.py#price_calculator`	原価積み上げ＋利益率で見積金額算出
4	`mfg/output_validator`	`quoting_pipeline.py#output_validator`	金額・工程データのバリデーション

1.7 法令・規格準拠ルール

法令	条文	適用箇所
下請代金支払遅延等防止法 第3条	書面の交付義務	見積書に記載すべき事項の網羅性チェック
下請代金支払遅延等防止法 第4条	支払遅延の禁止	見積書に支払条件を明記
独占禁止法（優越的地位の濫用）	不公正な取引方法の禁止	不当な値引き要求の検知
消費税法 第9条	適格請求書等保存方式	インボイス番号の記載（2023年10月～）

2. 生産計画AI（★キラーフィーチャー）

2.1 課題（金額換算の価値）

```
■ 直接効果①：納期遅延の防止
  納期遅延率 10% → 3% = 7%改善
  年間受注額5億 × 遅延によるペナルティ/信用毀損1% = 年500万円

■ 直接効果②：設備稼働率の向上
  稼働率 65% → 80% = 15%改善
  設備投資額2億 × 15% × 償却率10% = 年300万円の価値

■ 直接効果③：工場長の管理工数削減
  月40時間の計画立案 → 月10時間 = 30時間削減
  時給5,000円 × 30時間 = 月15万円 = 年180万円

■ 合計効果：年980万円
```

2.2 業務フロー

```
【日次】
  新規受注データ登録
    → ★AI Step 1: extractor（受注データ構造化）
    → ★AI Step 2: rule_matcher（工程マスタ照合）
    → ★AI Step 3: calculator（山積み計算）
    → ★AI Step 4: compliance（納期遵守チェック）
    → ★AI Step 5: generator（生産計画書PDF生成）
    → ★AI Step 6: validator（計画整合性チェック）
    → ★AI Step 7: saas_writer（execution_logs保存 + Slack通知）

【飛び込み受注時】
  新規受注シミュレーション
    → 「この案件を受けたら既存の納期に影響するか？」を即回答

【週次】
  設備稼働率レポート（投資判断の材料）
  ボトルネック設備の特定
```

2.3 パイプラインステップ定義

```
実装: workers/bpo/manufacturing/pipelines/production_planning_pipeline.py
エントリ: run_production_planning_pipeline(company_id, input_data)
ステップ数: 7
```

Step 1: extractor（受注データ構造化）

項目	内容
マイクロエージェント	`micro/structured_extractor`
入力	`orders[]` （product_name, quantity, delivery_date, processes[]）
処理	JSON直渡し or テキストからLLM抽出
出力	構造化された受注データリスト

Step 2: rule_matcher（工程マスタ照合）

項目	内容
マイクロエージェント	`micro/rule_matcher`
入力	全受注の工程リスト
処理	工程マスタとの照合。設備キャパシティの確認
出力	工程別の設備割り当て結果

工程マスタ（デフォルト）

```
PROCESS_MASTER = {
    "旋盤加工":   {"capacity_per_day": 8.0, "unit": "時間"},
    "フライス加工": {"capacity_per_day": 8.0, "unit": "時間"},
    "研磨加工":   {"capacity_per_day": 6.0, "unit": "時間"},
    "組立":       {"capacity_per_day": 10.0, "unit": "時間"},
    "溶接":       {"capacity_per_day": 8.0, "unit": "時間"},
    "検査":       {"capacity_per_day": 8.0, "unit": "時間"},
    "default":    {"capacity_per_day": 8.0, "unit": "時間"},
}
```

Step 3: calculator（山積み計算）

項目	内容
マイクロエージェント	`micro/cost_calculator`（calc_type= `production_load`）
入力	受注データ + 工程マスタ + 開始日
処理	山積み/山崩しアルゴリズム。ガントチャート生成
出力	`gantt[]`, `overloaded_processes[]`

山積み/山崩しアルゴリズム

```
山積み（Forward Scheduling）:
  ① 全受注を納期の早い順にソート
  ② 各受注の工程を設備に割り当て（設備×日の2次元マトリクス）
  ③ 各設備の日別負荷を累積
     負荷率 = Σ(受注工数) / 設備キャパ(h/日)
  ④ 負荷率 > 100% の日をオーバーロードとして記録

山崩し（Load Leveling）:
  ① オーバーロード日の受注を前倒し or 後ろ倒し
  ② 優先度: 納期厳守 > 得意先ランク > 利益率
  ③ 調整不能 → 外注提案 or 残業提案
  ④ 全日程の負荷率が100%以下になるまで繰り返し
```

設備稼働率計算式

```
稼働率(%) = 実稼働時間 / (計画時間 − 計画停止時間) × 100

  実稼働時間 = サイクルタイム × 良品数 + 段取り時間
  計画時間 = 1日8h × 稼働日数
  計画停止時間 = 定期メンテ + 昼休み + 朝礼

例: 月20日稼働、1日8h = 160h/月
  実稼働: 120h → 稼働率 75%
  目標: 80-85%（100%は非現実的。段取り替え・突発停止の余裕が必要）
```

Step 4: compliance（納期遵守チェック）

項目	内容
マイクロエージェント	`micro/compliance_checker`
入力	ガントチャート + 納期情報
処理	リードタイム逆算→納期遅延リスク判定
出力	`delivery_warnings[]`
アラート閾値	デフォルトリードタイム: 14日

Step 5: generator（生産計画書PDF生成）

項目	内容
マイクロエージェント	`micro/document_generator`
入力	ガントチャート + 受注データ + 警告
処理	テンプレート「生産計画書」で文書生成
出力	生産計画書（ガントチャート付き）

Step 6: validator（計画整合性チェック）

項目	内容
マイクロエージェント	`micro/output_validator`
入力	ガントチャート + キャパシティアラート
処理	設備稼働率100%超えアラート
出力	`capacity_alerts[]`
アラート閾値	`CAPACITY_ALERT_THRESHOLD = 1.0`（100%超え）

Step 7: saas_writer（execution_logs保存）

項目	内容
マイクロエージェント	`micro/saas_writer`
処理	execution_logs保存 + Slack通知（TODO）

2.4 マイクロエージェント命名マッピング

Step	マイクロエージェント	実装ファイル	責務
1	`mfg/pp_extractor`	`micro/extractor.py`	受注データ構造化
2	`mfg/pp_rule_matcher`	`micro/rule_matcher.py`	工程マスタ照合
3	`mfg/pp_calculator`	`micro/calculator.py`	山積み計算・ガントチャート生成
4	`mfg/pp_compliance`	インライン実装	納期遵守チェック
5	`mfg/pp_generator`	`micro/generator.py`	生産計画書PDF生成
6	`mfg/pp_validator`	`micro/validator.py`	設備稼働率100%超えアラート
7	`mfg/pp_writer`	インライン実装	execution_logs保存

2.5 法令・規格準拠ルール

法令	条文	適用箇所
労働基準法 第32条	労働時間（1日8時間、週40時間）	残業提案時の上限チェック
労働基準法 第36条	時間外・休日労働の協定（36協定）	残業上限: 月45h/年360h
労働安全衛生法 第66条の8の2	面接指導（月80時間超の時間外労働）	過重労働アラート

---

3. 品質管理

3.1 課題（金額換算の価値）

```
■ 直接効果①：不良率の低減
  不良率 3% → 1% = 年間売上5億 × 2%改善 = 年1,000万円

■ 直接効果②：品質クレーム対応工数削減
  月10件のクレーム対応 × 5時間/件 = 月50時間
  → 予兆検知で50%削減 = 月25時間 × ¥3,000 = 年90万円

■ 直接効果③：検査成績書の自動生成
  月20件 × 2時間/件 = 月40時間 → 月10時間に削減
  月30時間削減 × ¥3,000 = 年108万円
```

3.2 パイプラインステップ定義

```
実装: workers/bpo/manufacturing/pipelines/quality_control_pipeline.py
エントリ: run_quality_control_pipeline(company_id, input_data)
ステップ数: 7
```

Step 1: extractor（検査データ構造化）

項目	内容
マイクロエージェント	`micro/structured_extractor`
入力	`lot_number`, `product_name`, `measurements[]`, `usl`, `lsl`, `target`
処理	検査データの正規化・構造化
出力	構造化された検査データ

Step 2: calculator（SPC計算）

項目	内容
マイクロエージェント	`micro/cost_calculator`（calc_type=`spc`）
入力	測定値リスト + 規格上下限値
処理	Cp/Cpk/平均/標準偏差/管理限界の計算
出力	SPC計算結果
フォールバック	マイクロエージェント未対応時は `_calculate_spc()` で直接計算

3.3 SPC計算式の詳細

工程能力指数（Cp / Cpk）

```
# 工程能力指数 Cp: 規格幅と工程ばらつきの比
Cp = (USL - LSL) / (6 * sigma)
# Cp >= 1.33: 工程能力十分
# Cp >= 1.00: 工程能力あり（要改善）
# Cp <  1.00: 工程能力不足

# 工程能力指数 Cpk: 工程の偏りを考慮
Cpu = (USL - mu) / (3 * sigma)   # 上側工程能力
Cpl = (mu - LSL) / (3 * sigma)   # 下側工程能力
Cpk = min(Cpu, Cpl)
# Cpk >= 1.33: 偏りなく十分
# Cpk >= 1.00: 偏りあるが範囲内
# Cpk <  1.00: 規格外生産のリスクあり
```

```
# 実装: quality_control_pipeline.py#_calculate_spc
CP_WARNING_THRESHOLD = 1.0    # Cp < 1.0 は工程能力不足
CP_CAUTION_THRESHOLD = 1.33   # Cp < 1.33 は要改善
CPK_WARNING_THRESHOLD = 1.0   # Cpk < 1.0 は規格外リスク
```

Xbar-R管理図の管理限界計算

```
Xbar管理図（平均値管理図）:
  中心線 CL = X̄（全体平均）
  管理上限 UCL = X̄ + A2 × R̄
  管理下限 LCL = X̄ - A2 × R̄

R管理図（範囲管理図）:
  中心線 CL = R̄（範囲の平均）
  管理上限 UCL = D4 × R̄
  管理下限 LCL = D3 × R̄
```

A2, D3, D4 係数テーブル（サブグループサイズ別）

n（サブグループサイズ）	A2	D3	D4	d2
2	1.880	0	3.267	1.128
3	1.023	0	2.575	1.693
4	0.729	0	2.282	2.059
5	0.577	0	2.115	2.326
6	0.483	0	2.004	2.534
7	0.419	0.076	1.924	2.704
8	0.373	0.136	1.864	2.847
9	0.337	0.184	1.816	2.970
10	0.308	0.223	1.777	3.078

```
# 現在の実装（簡易版: 3σ法）
ucl = mean + 3 * std
lcl = mean - 3 * std

# Phase 2+で正式なA2/D3/D4係数テーブルによる計算に拡張
```

品質要求レベルによる見積影響

レベル	公差	倍率	検査方法
Q1（一般）	±0.1mm	1.00倍	抜き取り検査
Q2（精密）	±0.02mm	1.19倍	全数検査 + 検査成績表
Q3（高精密）	±0.005mm	1.66倍	研磨 + 3次元測定 + SPC
Q4（超精密/規制品）	±0.001mm	2.25倍	書類だけで見積の15-30%

Step 3: rule_matcher（管理限界逸脱チェック）

項目	内容
マイクロエージェント	`micro/rule_matcher`（rule_type= `control_limit` ）
入力	測定値リスト + UCL/LCL
処理	管理限界超え検出 + Western Electric Rules（連続7点上昇等）
出力	`violations[]`（逸脱点のインデックスと値）

Step 4: compliance（ISO 9001要求事項照合）

項目	内容
マイクロエージェント	`micro/compliance_checker`
入力	全ステップのデータ
処理	ISO 9001必須チェック項目の確認
出力	`iso9001_warnings[]`

```
ISO9001_REQUIRED_CHECKS = [
    "測定データ記録",       # ISO 9001: 9.1.1 監視及び測定
    "規格値設定",           # ISO 9001: 8.6 製品及びサービスのリリース
    "管理図更新",           # ISO 9001: 9.1.3 分析及び評価
    "不適合品処置",         # ISO 9001: 10.2 不適合及び是正処置
]
```

Step 5-7: generator / validator / saas_writer

Step	処理	出力
5	品質月次レポート生成	SPC結果 + 逸脱 + ISO警告を含むレポート
6	不良予兆検知（トレンド分析）	`trend_alerts[]`（Cp/Cpk低下、管理限界逸脱数）
7	品質記録保存 + アラート通知	execution_logs

3.4 マイクロエージェント命名マッピング

Step	マイクロエージェント	実装ファイル	責務
1	`mfg/qc_extractor`	`micro/extractor.py`	検査データ構造化
2	`mfg/qc_calculator`	`micro/calculator.py` + フォールバック	SPC計算
3	`mfg/qc_rule_matcher`	`micro/rule_matcher.py`	管理限界逸脱チェック
4	`mfg/qc_compliance`	インライン実装	ISO 9001チェック
5	`mfg/qc_generator`	`micro/generator.py`	品質月次レポート生成
6	`mfg/qc_validator`	`micro/validator.py`	不良予兆検知
7	`mfg/qc_writer`	インライン実装	品質記録保存

3.5 法令・規格準拠ルール

法令・規格	条項	適用箇所
ISO 9001:2015 8.5.1	製造及びサービス提供の管理	工程管理計画（QC工程表）
ISO 9001:2015 8.6	製品及びサービスのリリース	出荷判定基準
ISO 9001:2015 9.1.1	監視、測定、分析及び評価	SPC管理図の運用
ISO 9001:2015 9.1.3	分析及び評価	品質月次レポート
ISO 9001:2015 10.2	不適合及び是正処置	不適合品処理フロー
IATF 16949 8.6.2	レイアウト検査	自動車部品の初回品検査（S4セグメント）
JIS Z 9020-2	管理図	Xbar-R管理図の運用基準
JIS Z 9021	工程能力指数	Cp/Cpkの算出方法
製造物責任法（PL法）第3条	製造物の欠陥	トレーサビリティ記録の保持

---

4. 在庫最適化

4.1 課題（金額換算の価値）

```
■ 直接効果①：在庫金額の削減
  在庫金額3,000万円 × 20%削減 = 600万円のキャッシュ改善

■ 直接効果②：欠品によるライン停止防止
  月1回の欠品停止 × 半日 × 日当15万円 = 年90万円

■ 直接効果③：発注業務の効率化
  月20時間の発注管理 → 月5時間 = 月15時間削減
  × ¥3,000 = 年54万円
```

4.2 パイプラインステップ定義

```
実装: workers/bpo/manufacturing/pipelines/inventory_optimization_pipeline.py
エントリ: run_inventory_optimization_pipeline(company_id, input_data)
ステップ数: 7
```

Step 1: saas_reader（在庫・入出庫データ取得）

項目	内容
マイクロエージェント	`micro/saas_reader`
入力	`items[]` （item_code, item_name, current_stock, unit_price, lead_time_days, usage_history[]）
処理	DB/SaaSから在庫データ取得（Phase 1は直渡し）

Step 2: extractor（需要パターン構造化）

項目	内容
マイクロエージェント	`micro/structured_extractor`
入力	在庫品目リスト
処理	需要パターンの構造化

Step 3: calculator（ABC分析 + 安全在庫 + 発注点）

項目	内容
マイクロエージェント	`micro/cost_calculator` （calc_type= `inventory_optimization` ）
入力	品目リスト
処理	ABC分析、安全在庫計算、EOQ計算、発注点算出
フォールバック	`_calculate_inventory_optimization()`

4.3 在庫計算式の詳細

安全在庫

```
安全在庫 = k × sigma_d × sqrt(LT)

  k:       安全係数（サービス率に対応）
  sigma_d: 需要の標準偏差（月間使用量のばらつき）
  LT:      リードタイム（月単位）
```

サービス率	安全係数 k	欠品確率
90%	1.28	10%
95%	1.645	5%
99%	2.33	1%
99.9%	3.09	0.1%

```
# 実装: inventory_optimization_pipeline.py
SAFETY_FACTOR_Z = 1.645  # 95%サービス率

safety_stock = SAFETY_FACTOR_Z * std_usage * math.sqrt(lead_time_months)
```

発注点（Reorder Point）

```
発注点 = 平均月間使用量 × リードタイム(月) + 安全在庫

例: 平均月間使用量100kg、リードタイム14日（0.47月）、sigma_d=20kg
  安全在庫 = 1.645 × 20 × sqrt(0.47) = 22.5kg
  発注点 = 100 × 0.47 + 22.5 = 69.5kg
  → 在庫が69.5kgを下回ったら発注
```

経済的発注量（EOQ）

```
EOQ = sqrt(2 × D × S / H)

  D: 年間需要量
  S: 1回の発注コスト
  H: 年間保管コスト（単価 × 保管コスト率）
```

```
# 実装: inventory_optimization_pipeline.py
ordering_cost = unit_price * 0.1    # 発注コスト = 単価×10%（簡易）
holding_cost = unit_price * 0.2     # 保管コスト = 単価×20%（簡易）
eoq = math.sqrt(2 * annual_usage * ordering_cost / max(holding_cost, 0.001))
```

ABC分析

クラス	累積使用金額比率	管理レベル	発注方式
A	0-70%	定量発注（厳密管理）	発注点方式 + 安全在庫
B	70-95%	定期発注（中程度管理）	定期不定量発注
C	95-100%	目視管理（簡易管理）	2ビン方式 or まとめ発注

```
# 実装: inventory_optimization_pipeline.py
ABC_A_THRESHOLD = 0.70   # 累積70%がAクラス
ABC_B_THRESHOLD = 0.95   # 累積95%までがBクラス（残りがC）
```

在庫回転率

```
在庫回転率（回/年） = 年間消費金額 / 平均在庫金額
```

```
# 実装: inventory_optimization_pipeline.py
TURNOVER_RATE_WARNING = 6.0    # 6回/年未満 = 過剰在庫傾向
TURNOVER_RATE_CAUTION = 12.0   # 12回/年未満 = 要改善
# 製造業平均: 8-15回/年
```

Step 4-7: rule_matcher / generator / validator / saas_writer

Step	処理	出力
4	発注点アラート判定（current_stock <= reorder_point）	`order_alerts[]`
5	発注推奨リスト生成	ABC分析結果 + 発注推奨
6	在庫回転率チェック	`turnover_warnings[]`
7	発注推奨保存 + Slack通知	execution_logs

4.4 マイクロエージェント命名マッピング

Step	マイクロエージェント	実装ファイル	責務
1	`mfg/inv_reader`	インライン実装	在庫・入出庫データ取得
2	`mfg/inv_extractor`	`micro/extractor.py`	需要パターン構造化
3	`mfg/inv_calculator`	`micro/calculator.py` + フォールバック	ABC分析 + 安全在庫 + EOQ
4	`mfg/inv_rule_matcher`	`micro/rule_matcher.py`	発注点アラート判定
5	`mfg/inv_generator`	`micro/generator.py`	発注推奨リスト生成
6	`mfg/inv_validator`	`micro/validator.py`	在庫回転率チェック
7	`mfg/inv_writer`	インライン実装	発注推奨保存

4.5 法令・規格準拠ルール

法令	条文	適用箇所
棚卸資産の評価に関する会計基準	企業会計基準第9号	在庫評価方法（最終仕入原価法 or 移動平均法）
法人税法施行令 第28条	棚卸資産の評価方法	税務上の在庫評価
ISO 9001:2015 8.5.4	保存	在庫品の劣化管理（先入先出）

---

5. SOP管理（標準作業手順書）

5.1 課題（金額換算の価値）

```
■ 直接効果①：手順書作成工数の削減
  新規SOP作成: 1件8時間 × 月2件 = 月16時間
  → AI生成で月4時間に削減 = 年14.4万円

■ 直接効果②：作業ミスの減少
  手順書不備による不良: 月2件 × 対応コスト5万円 = 年120万円
  → 50%削減 = 年60万円

■ 直接効果③：外国人実習生の教育効率化
  OJT時間: 月40時間 → 多言語SOPで月25時間
  月15時間 × ¥3,000 = 年54万円
```

5.2 パイプラインステップ定義

```
実装: workers/bpo/manufacturing/pipelines/sop_management_pipeline.py
エントリ: run_sop_management_pipeline(company_id, input_data, existing_sop_id)
ステップ数: 7
```

Step 1: extractor（テキスト抽出・構造化）

項目	内容
マイクロエージェント	`micro/document_ocr` + `micro/structured_extractor`
入力	`text`（直接入力） or `file_path`（テキスト/画像ファイル）
処理	OCRテキスト抽出→LLMで構造化（title, steps[], materials[], tools[]）
出力	構造化されたSOP元データ

Step 2: generator（SOP生成）

項目	内容
マイクロエージェント	`micro/document_generator`
入力	構造化データ + テンプレート「標準作業手順書（SOP）」
処理	LLMでSOP文書を生成
出力	SOP文書（タイトル、バージョン、手順ステップ、安全注意事項）

Step 3: compliance（安全衛生法チェック）

項目	内容
マイクロエージェント	`micro/compliance_checker`
入力	生成されたSOP文書
処理	安全衛生法必須記載事項の存在チェック
出力	`compliance_warnings[]`

安全衛生法 必須記載事項

```
SAFETY_REQUIRED_ITEMS = [
    "保護具の着用",   # 労働安全衛生規則第593-598条
    "緊急時対応",     # 労働安全衛生法第25条（緊急時の措置）
    "作業前確認",     # 労働安全衛生規則第151条の3
    "作業後処置",     # 労働安全衛生規則第619-625条
]
```

Step 4: diff（既存SOPとの差分検出）

項目	内容
マイクロエージェント	`micro/diff_checker`
入力	新SOP + 既存SOP（existing_sop_id指定時）
処理	改訂箇所の特定、変更理由の記録
出力	`diff_result`（has_diff, changes[], revision_reason）

Step 5-7: validator / generator_pdf / saas_writer

Step	処理	出力
5	手順の論理整合性チェック（必須フィールド確認）	検証結果
6	PDF/HTML出力	印刷可能なSOP文書
7	SOP保存 + 承認フロー開始（TODO）	execution_logs

5.3 マイクロエージェント命名マッピング

Step	マイクロエージェント	実装ファイル	責務
1	`mfg/sop_extractor`	`micro/ocr.py` + `micro/extractor.py`	テキスト抽出・構造化
2	`mfg/sop_generator`	`micro/generator.py`	SOP文書生成
3	`mfg/sop_compliance`	インライン実装	安全衛生法チェック
4	`mfg/sop_diff`	インライン実装	既存SOPとの差分検出
5	`mfg/sop_validator`	`micro/validator.py`	論理整合性チェック
6	`mfg/sop_pdf_generator`	`micro/generator.py`	PDF/HTML出力
7	`mfg/sop_writer`	インライン実装	SOP保存 + 承認フロー

5.4 法令・規格準拠ルール

法令	条文	適用箇所
労働安全衛生法 第20条	事業者の講ずべき措置（機械、器具等による危険の防止）	設備操作手順の安全措置記載
労働安全衛生法 第22条	健康障害の防止措置	有害物質取扱いの保護措置
労働安全衛生法 第25条	緊急時の措置	緊急時対応手順の記載
労働安全衛生法 第59条	安全衛生教育	新人教育用SOPの整備
労働安全衛生規則 第593-598条	保護具の着用	保護具着用指示の記載
ISO 9001:2015 7.5	文書化した情報	SOP文書の版管理
ISO 9001:2015 8.5.1	製造及びサービス提供の管理	作業指示書の整備

---

6. 設備保全

6.1 課題（金額換算の価値）

```
■ 直接効果①：突発停止の防止
  月2回の突発停止 × 半日の損失 × 日当20万円 = 年480万円
  → 予防保全で50%削減 = 年240万円

■ 直接効果②：法定点検の漏れ防止
  ボイラー/クレーン/プレスの法定点検漏れ → 行政処分リスク回避

■ 直接効果③：保全コストの最適化
  事後保全（壊れてから直す）→ 予防保全（壊れる前に直す）
  修理費: 予防保全は事後保全の1/3-1/5
```

6.2 パイプラインステップ定義

```
実装: workers/bpo/manufacturing/pipelines/equipment_maintenance_pipeline.py
エントリ: run_equipment_maintenance_pipeline(company_id, input_data, target_month)
ステップ数: 7
```

Step 1: saas_reader（設備マスタ + 保全履歴取得）

項目	内容
マイクロエージェント	`micro/saas_reader`
入力	`equipments[]` （equipment_id, name, type, last_maintenance_date, interval_days, failure_history[]）
処理	DB/SaaSから設備データ取得（Phase 1は直渡し）

Step 2: extractor（保全記録構造化）

項目	内容
マイクロエージェント	`micro/structured_extractor`
入力	設備マスタ + 保全履歴
処理	保全記録の構造化

Step 3: calculator（MTBF/MTTR計算 + 次回保全日算出）

項目	内容
マイクロエージェント	`micro/cost_calculator` （calc_type= `mtbf_mttr` ）
入力	設備データ + 故障履歴
処理	MTBF/MTTR計算、次回保全日算出
フォールバック	`_calculate_mtbf_mttr()`

6.3 MTBF/MTTR/OEE計算式の詳細

MTBF（Mean Time Between Failures: 平均故障間隔）

```
MTBF = 総稼働時間 / 故障回数

例: 稼働時間10,000h、故障5回
  MTBF = 10,000 / 5 = 2,000h（約83日）
  → 「平均して2,000時間に1回壊れる」
```

MTTR（Mean Time To Repair: 平均修理時間）

```
MTTR = 総修理時間 / 故障回数

例: 修理合計20h、故障5回
  MTTR = 20 / 5 = 4h
  → 「壊れたら平均4時間で直る」
```

```
# 実装: equipment_maintenance_pipeline.py#_calculate_mtbf_mttr
MIN_SAMPLE_FOR_MTBF = 2  # MTBF算出に最低2件の故障データが必要

if len(failure_history) >= MIN_SAMPLE_FOR_MTBF:
    total_repair_hours = sum(f["repair_hours"] for f in failure_history)
    failure_count = len(failure_history)
    mttr = total_repair_hours / failure_count
    mtbf = (operating_hours - total_repair_hours) / failure_count
```

設備総合効率（OEE: Overall Equipment Effectiveness）

```
OEE = 時間稼働率 × 性能稼働率 × 良品率

  時間稼働率 = (負荷時間 − 停止時間) / 負荷時間
    負荷時間 = 就業時間 − 計画停止時間
    停止時間 = 故障停止 + 段取り替え + 調整

  性能稼働率 = (基準サイクルタイム × 加工数量) / 稼働時間
    速度低下・チョコ停を反映

  良品率 = 良品数 / 加工数量
    不良・手直しを反映

世界標準:
  OEE 85%以上 = ワールドクラス
  OEE 60-85% = 一般的
  OEE 60%以下 = 改善余地大

例: 負荷時間480min, 停止60min, 基準CT1min, 加工数380個, 良品370個
  時間稼働率 = (480-60)/480 = 87.5%
  性能稼働率 = (1×380)/(480-60) = 90.5%
  良品率 = 370/380 = 97.4%
  OEE = 87.5% × 90.5% × 97.4% = 77.1%
```

次回保全日算出

```
next_maintenance_date = last_maintenance_date + timedelta(days=maintenance_interval_days)
```

Step 4: rule_matcher（保全期限アラート判定）

項目	内容
マイクロエージェント	`micro/rule_matcher`（rule_type= `maintenance_deadline` ）
入力	設備統計 + 保全期限
処理	保全期限アラート判定（通常30日前、法定点検60日前）
出力	`maintenance_alerts[]`（severity: overdue/warning）

```
MAINTENANCE_ALERT_DAYS = 30              # 30日以内に期限
MANDATORY_INSPECTION_ALERT_DAYS = 60     # 法定点検は60日前からアラート
```

Step 5-7: generator / validator / saas_writer

Step	処理	出力
5	月次保全カレンダー生成	設備別の保全スケジュール
6	保全計画の完全性チェック（情報不完全な設備を検出）	`missing_equipments[]`
7	保全計画保存 + リマインダー登録	execution_logs

6.4 マイクロエージェント命名マッピング

Step	マイクロエージェント	実装ファイル	責務
1	`mfg/em_reader`	インライン実装	設備マスタ + 保全履歴取得
2	`mfg/em_extractor`	`micro/extractor.py`	保全記録構造化
3	`mfg/em_calculator`	`micro/calculator.py` + フォールバック	MTBF/MTTR計算
4	`mfg/em_rule_matcher`	`micro/rule_matcher.py`	保全期限アラート判定
5	`mfg/em_generator`	`micro/generator.py`	月次保全カレンダー生成
6	`mfg/em_validator`	`micro/validator.py`	保全計画の完全性チェック
7	`mfg/em_writer`	インライン実装	保全計画保存 + リマインダー

6.5 法令・規格準拠ルール

法令	条文	適用箇所
労働安全衛生法 第45条	定期自主検査	プレス機械/クレーン等の年次検査
労働安全衛生法施行令 第15条	特定機械等の検査	ボイラー/第一種圧力容器の性能検査
クレーン等安全規則 第34条	定期自主検査（年次）	クレーンの年次検査
ボイラー及び圧力容器安全規則 第32条	定期自主検査	ボイラーの月次点検
消防法 第17条の3の3	消防用設備等の点検	消火設備の半年/年次点検
電気事業法 第42条	保安規程	自家用電気工作物の保安点検
ISO 9001:2015 7.1.3	インフラストラクチャ	設備の維持管理計画

---

7. 仕入管理（MRP対応）

7.1 課題（金額換算の価値）

```
■ 直接効果①：欠品防止
  月1回の材料欠品 × ライン停止半日 × 日当15万円 = 年90万円

■ 直接効果②：発注コストの最適化
  過剰発注による在庫金利コスト: 年50万円の削減

■ 直接効果③：下請法コンプライアンス
  支払遅延による行政処分リスクの回避
```

7.2 パイプラインステップ定義

```
実装: workers/bpo/manufacturing/pipelines/procurement_pipeline.py
エントリ: run_procurement_pipeline(company_id, input_data)
ステップ数: 7
```

Step 1: extractor（BOM構造化）

項目	内容
マイクロエージェント	`micro/structured_extractor`
入力	`production_order` （product_code, quantity, required_date） + `bom[]` （part_code, quantity_per_unit, current_stock, pending_orders, unit_price, lead_time_days, payment_terms_days）
処理	BOM（部品表）の構造化
出力	構造化されたBOMデータ

Step 2: calculator（MRP所要量計算）

項目	内容
マイクロエージェント	`micro/cost_calculator` （calc_type= `mrp` ）
入力	生産指示 + BOM
処理	正味所要量計算（MRP展開）
フォールバック	`_calculate_mrp()`

7.3 MRP計算式の詳細

正味所要量計算

```
正味所要量 = 総所要量 − 手持在庫 − 発注残（注文済み未入庫）

  総所要量 = 員数（quantity_per_unit） × 生産指示数量

例: 製品100個生産、ボルトM8×20の員数4本/個
  総所要量 = 4 × 100 = 400本
  手持在庫 = 150本
  発注残 = 100本
  正味所要量 = 400 − 150 − 100 = 150本 → 要発注
```

```
# 実装: procurement_pipeline.py#_calculate_mrp
gross_requirement = qty_per_unit * order_qty
net_requirement = max(0, gross_requirement - current_stock - pending_orders)
```

MRP展開のタイムバケット

```
 週1  │  週2  │  週3  │  週4  │  週5
──────┼──────┼──────┼──────┼──────
   ↑                     ↑      ↑
  今日                 発注日   必要日

  発注日 = 必要日 − リードタイム
  → リードタイムを逆算して発注日を確定

BOM展開（多段階）:
  完成品A
    ├─ 部品B (×2)
    │    ├─ 材料D (×0.5kg)
    │    └─ 材料E (×3個)
    └─ 部品C (×1)
         └─ 材料F (×2kg)

  → 部品B/Cの所要量を先に計算
  → その結果から材料D/E/Fの所要量を計算（多段展開）
```

Step 3: rule_matcher（発注先選定）

項目	内容
マイクロエージェント	`micro/rule_matcher`（rule_type=`supplier_selection`）
入力	発注所要量リスト
処理	発注先のQCD（品質/コスト/納期）評価に基づく選定
出力	推奨発注先の決定

Step 4: compliance（下請法チェック）

項目	内容
マイクロエージェント	`micro/compliance_checker`
入力	発注リスト（支払条件含む）
処理	下請法の支払期日チェック（60日ルール）
出力	`compliance_warnings[]`

7.4 下請法準拠ルール

条文	内容	AI対応
第3条（書面交付義務）	発注時に以下を記載した書面を交付: 下請事業者の名称、製造委託の内容、下請代金の額、支払期日、支払方法	発注書PDFに必須項目の自動記載チェック
第4条1項2号（支払遅延の禁止）	受領日から60日以内の支払期日を定めること。60日を超える支払条件はNG	`SUBCONTRACT_PAYMENT_MAX_DAYS = 60` で自動チェック
第4条1項3号（減額の禁止）	下請代金の額を減ずること（不当な値引き要求）の禁止	前回発注額との乖離を検出（±30%超でアラート）
第4条1項5号（買いたたきの禁止）	通常支払われる対価に比べ著しく低い額を不当に定めること	市場単価DBとの比較チェック
第4条2項1号（有償支給原材料の早期決済の禁止）	有償支給材の対価を下請代金の支払期日前に支払わせること	有償支給フラグ時の支払日チェック

```
# 実装: procurement_pipeline.py
SUBCONTRACT_PAYMENT_MAX_DAYS = 60
UNIT_PRICE_VARIANCE_THRESHOLD = 0.30  # ±30%以上の価格差はアラート
```

Step 5-7: generator / validator / saas_writer

Step	処理	出力
5	発注書PDF生成（下請法第3条準拠）	発注書PDF
6	発注金額・納期の妥当性チェック	`validity_warnings[]`
7	発注記録保存 + 仕入先通知（TODO）	execution_logs

7.5 マイクロエージェント命名マッピング

Step	マイクロエージェント	実装ファイル	責務
1	`mfg/proc_extractor`	`micro/extractor.py`	BOM構造化
2	`mfg/proc_calculator`	`micro/calculator.py` + フォールバック	MRP所要量計算
3	`mfg/proc_rule_matcher`	`micro/rule_matcher.py`	発注先選定
4	`mfg/proc_compliance`	インライン実装	下請法チェック
5	`mfg/proc_generator`	`micro/generator.py`	発注書PDF生成
6	`mfg/proc_validator`	`micro/validator.py`	発注金額・納期チェック
7	`mfg/proc_writer`	インライン実装	発注記録保存

7.6 法令・規格準拠ルール

法令	条文	適用箇所
下請代金支払遅延等防止法 第3条	書面の交付義務	発注書必須記載事項チェック
下請代金支払遅延等防止法 第4条1項2号	支払遅延の禁止（60日ルール）	支払期日の自動チェック
下請代金支払遅延等防止法 第4条1項3号	減額の禁止	不当な値引き検出
下請代金支払遅延等防止法 第4条1項5号	買いたたきの禁止	市場単価との比較
下請代金支払遅延等防止法 第5条	書類の作成・保存義務（2年間）	発注書の保存管理
消費税法 第30条	仕入税額控除	インボイス番号の確認

---

8. ISO文書管理

8.1 課題（金額換算の価値）

```
■ 直接効果①：ISO審査準備の工数削減
  審査前準備: 100時間/年 → 30時間/年 = 70時間削減
  × ¥3,000 = 年21万円

■ 直接効果②：不適合指摘の予防
  軽微な不適合1件の対応コスト: 10万円
  年3件の不適合を予防 = 年30万円

■ 直接効果③：認証失効リスクの回避
  ISO認証の維持は取引条件。失効 = 取引停止リスク
```

8.2 パイプラインステップ定義

```
実装: workers/bpo/manufacturing/pipelines/iso_document_pipeline.py
エントリ: run_iso_document_pipeline(company_id, input_data, iso_standard, previous_audit_id)
ステップ数: 8
```

Step 1: saas_reader（文書マスタ + 改訂履歴取得）

項目	内容
マイクロエージェント	`micro/saas_reader`
入力	`documents[]`（document_id, name, type, version, last_revised_date, expiry_date, department, iso_clause）
処理	DB/SaaSから文書データ取得（Phase 1は直渡し）

Step 2: extractor（文書メタデータ構造化）

項目	内容
マイクロエージェント	`micro/structured_extractor`
入力	文書リスト
処理	文書メタデータの構造化

Step 3: rule_matcher（ISO条項別の文書有無チェック）

項目	内容
マイクロエージェント	`micro/rule_matcher`（rule_type=`iso_clause_coverage`）
入力	文書リスト + ISO必須文書リスト
処理	必須文書の欠損検出
出力	`missing_mandatory[]`

8.3 ISO 9001:2015 条項マッピング

必須文書（文書化した情報として要求されるもの）

条項	内容	必須文書
4.3	品質マネジメントシステムの適用範囲	適用範囲文書
4.4	品質マネジメントシステム及びそのプロセス	品質マニュアル（推奨）
5.2	品質方針	品質方針文書
6.2	品質目標及びそれを達成するための計画策定	品質目標文書
7.1.5	監視及び測定のための資源	校正記録
7.2	力量	力量の証拠（教育訓練記録）
7.5	文書化した情報	文書管理手順書 + 記録管理手順書
8.1	運用の計画及び管理	QC工程表/作業手順書
8.2.3	製品及びサービスに関する要求事項のレビュー	契約レビュー記録
8.3.3	設計・開発へのインプット	設計入力記録（該当する場合）
8.4	外部から提供されるプロセス、製品及びサービスの管理	購買仕様書/受入検査記録
8.5.1	製造及びサービス提供の管理	作業指示書(SOP)
8.5.2	識別及びトレーサビリティ	トレーサビリティ記録
8.6	製品及びサービスのリリース	出荷検査記録
8.7	不適合なアウトプットの管理	不適合品処理記録
9.1.1	監視、測定、分析及び評価	品質データ分析記録
9.2	内部監査	内部監査手順書 + 監査記録 + 是正処置記録
9.3	マネジメントレビュー	マネジメントレビュー記録
10.2	不適合及び是正処置	是正処置手順書 + 是正処置記録

```
# 実装: iso_document_pipeline.py
ISO9001_MANDATORY_DOCUMENTS = [
    "品質マニュアル",
    "品質方針",
    "品質目標",
    "文書管理手順書",
    "記録管理手順書",
    "内部監査手順書",
    "不適合管理手順書",
    "是正処置手順書",
]

ISO14001_MANDATORY_DOCUMENTS = [
    "環境マニュアル",
    "環境方針",
    "環境目標",
    "緊急事態対応手順書",
]
```

Step 4: compliance（有効期限・改訂周期チェック）

項目	内容
マイクロエージェント	`micro/compliance_checker`
入力	文書リスト + 期限情報
処理	有効期限チェック + 改訂周期チェック
出力	`expiry_alerts[]` , `revision_alerts[]`

```
DOCUMENT_EXPIRY_ALERT_DAYS = 60        # 有効期限60日前からアラート
DEFAULT_REVISION_CYCLE_YEARS = 3       # デフォルト改訂周期3年
```

Step 5: diff（前回監査との差分検出）

項目	内容
マイクロエージェント	`micro/diff_checker`
入力	現在の文書体系 + 前回監査データ（previous_audit_id指定時）
処理	前回監査からの変更点の特定

Step 6: generator（監査チェックリスト + 不適合レポート生成）

項目	内容
マイクロエージェント	`micro/document_generator`
テンプレート	「ISO監査チェックリスト」
入力	ISO規格 + 文書一覧 + 欠損文書 + 期限アラート + 差分情報
出力	監査チェックリスト文書

Step 7: validator（文書体系の完全性チェック）

項目	内容
マイクロエージェント	`micro/output_validator`
入力	文書一覧 + 欠損文書 + 期限切れ文書
処理	必須文書の欠損 + 期限切れ文書を `completeness_issues[]` として集約

Step 8: saas_writer（監査記録保存 + 期限アラート）

項目	内容
マイクロエージェント	`micro/saas_writer`
処理	監査記録保存（TODO: iso_audit_records テーブル）+ 期限アラートメール

8.4 マイクロエージェント命名マッピング

Step	マイクロエージェント	実装ファイル	責務
1	`mfg/iso_reader`	インライン実装	文書マスタ + 改訂履歴取得
2	`mfg/iso_extractor`	`micro/extractor.py`	文書メタデータ構造化
3	`mfg/iso_rule_matcher`	`micro/rule_matcher.py`	ISO条項別文書チェック
4	`mfg/iso_compliance`	インライン実装	有効期限・改訂周期チェック
5	`mfg/iso_diff`	インライン実装	前回監査との差分検出
6	`mfg/iso_generator`	`micro/generator.py`	監査チェックリスト生成
7	`mfg/iso_validator`	`micro/validator.py`	文書体系の完全性チェック
8	`mfg/iso_writer`	インライン実装	監査記録保存

8.5 法令・規格準拠ルール

法令・規格	条項	適用箇所
ISO 9001:2015 4.4	品質マネジメントシステム及びそのプロセス	プロセスの特定・相互関係の文書化
ISO 9001:2015 7.5	文書化した情報	文書の作成・更新・管理の手順
ISO 9001:2015 8.5	製造及びサービス提供	作業手順書・QC工程表の管理
ISO 9001:2015 9.2	内部監査	監査計画・監査記録・是正処置
ISO 9001:2015 10.2	不適合及び是正処置	不適合管理・原因分析・再発防止
ISO 14001:2015 6.1.2	環境側面	環境影響評価の文書化
ISO 14001:2015 8.1	運用の計画及び管理	環境管理手順の文書化
IATF 16949 7.5	文書化した情報（自動車業界追加要求）	PPAP/APQP文書の管理（S4セグメント）
JISQ 9100 7.5	文書化した情報（航空宇宙追加要求）	特殊工程記録の管理（S6セグメント）

データモデル（共通）

DB設計

```
-- 見積マスタ
CREATE TABLE mfg_quotes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    quote_number TEXT NOT NULL,                 -- 見積番号（採番ルール: QT-YYYYMMDD-NNN）
    product_name TEXT NOT NULL,                 -- 製品名
    material TEXT NOT NULL,                     -- 材質（SS400/SUS304等）
    quantity INTEGER NOT NULL,                  -- 数量
    order_type TEXT DEFAULT 'standard',         -- standard/rush/prototype/large_lot
    delivery_days INTEGER,                      -- 納期（日数）
    total_material_cost INTEGER DEFAULT 0,      -- 材料費合計（円）
    total_processing_cost INTEGER DEFAULT 0,    -- 加工費合計（円）
    overhead_cost INTEGER DEFAULT 0,            -- 管理費（円）
    profit_rate DECIMAL(4,3),                   -- 利益率
    total_amount INTEGER NOT NULL,              -- 見積総額（円）
    confidence DECIMAL(3,2),                    -- AI信頼度（0.00-1.00）
    customer_id UUID,                           -- 得意先ID
    customer_name TEXT,                         -- 得意先名
    status TEXT DEFAULT 'draft',                -- draft/submitted/won/lost/expired
    pipeline_result JSONB,                      -- パイプライン実行結果（全ステップ）
    submitted_at TIMESTAMPTZ,
    expired_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 見積明細（工程別）
CREATE TABLE mfg_quote_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    quote_id UUID NOT NULL REFERENCES mfg_quotes(id),
    process_name TEXT NOT NULL,                 -- 工程名
    hourly_rate INTEGER NOT NULL,               -- チャージレート（円/h）
    estimated_hours DECIMAL(6,2),               -- 加工時間（h）
    setup_hours DECIMAL(6,2),                   -- 段取り時間（h）
    processing_cost INTEGER NOT NULL,           -- 加工費（円）
    sort_order INTEGER DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- 生産計画
CREATE TABLE mfg_production_plans (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    plan_month TEXT NOT NULL,                   -- YYYY-MM
    start_date DATE NOT NULL,
    gantt_data JSONB NOT NULL,                  -- ガントチャート（JSON）
    capacity_alerts JSONB DEFAULT '[]',         -- 稼働率超過アラート
    delivery_warnings JSONB DEFAULT '[]',       -- 納期遅延警告
    status TEXT DEFAULT 'draft',                -- draft/confirmed/completed
    pipeline_result JSONB,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 品質検査記録
CREATE TABLE mfg_quality_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    lot_number TEXT NOT NULL,                   -- ロット番号
```

```
    product_name TEXT NOT NULL,
    report_month TEXT,                          -- YYYY-MM
    measurements JSONB NOT NULL,                -- 測定値リスト
    usl DECIMAL(12,4),                          -- 規格上限値
    lsl DECIMAL(12,4),                          -- 規格下限値
    target_value DECIMAL(12,4),                 -- 規格中心値
    cp DECIMAL(6,3),                            -- 工程能力指数
    cpk DECIMAL(6,3),                           -- Cpk
    mean_value DECIMAL(12,4),                   -- 平均値
    std_value DECIMAL(12,4),                    -- 標準偏差
    ucl DECIMAL(12,4),                          -- 管理上限
    lcl DECIMAL(12,4),                          -- 管理下限
    violations JSONB DEFAULT '[]',              -- 管理限界逸脱
    trend_alerts JSONB DEFAULT '[]',            -- 不良予兆アラート
    pipeline_result JSONB,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- 在庫マスタ
CREATE TABLE mfg_inventory_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    item_code TEXT NOT NULL,                    -- 品目コード
    item_name TEXT NOT NULL,
    item_type TEXT DEFAULT 'material',          -- material/wip/finished/consumable
    unit TEXT DEFAULT '個',                     -- 単位
    unit_price DECIMAL(12,2),                   -- 単価
    current_stock DECIMAL(12,2) DEFAULT 0,      -- 現在在庫数
    safety_stock DECIMAL(12,2) DEFAULT 0,       -- 安全在庫
    reorder_point DECIMAL(12,2) DEFAULT 0,      -- 発注点
    lead_time_days INTEGER DEFAULT 14,          -- リードタイム（日）
    abc_class TEXT DEFAULT 'C',                 -- A/B/C
    preferred_supplier TEXT,                    -- 主要仕入先
    last_order_date DATE,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 設備マスタ
CREATE TABLE mfg_equipments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    equipment_code TEXT NOT NULL,               -- 設備コード
    equipment_name TEXT NOT NULL,
    equipment_type TEXT NOT NULL,               -- 旋盤/MC/研磨/プレス等
    charge_rate INTEGER,                        -- チャージレート（円/h）
    capacity_hours_per_day DECIMAL(4,1) DEFAULT 8.0,
    last_maintenance_date DATE,
    maintenance_interval_days INTEGER DEFAULT 90,
    is_mandatory_inspection BOOLEAN DEFAULT false,  -- 法定点検対象
    operating_hours DECIMAL(10,1) DEFAULT 0,    -- 累計稼働時間
    mtbf_hours DECIMAL(10,1),                   -- MTBF
    mttr_hours DECIMAL(6,1),                    -- MTTR
    status TEXT DEFAULT 'active',               -- active/maintenance/retired
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 設備故障履歴
CREATE TABLE mfg_equipment_failures (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    equipment_id UUID NOT NULL REFERENCES mfg_equipments(id),
    failure_date DATE NOT NULL,
    repair_hours DECIMAL(6,1) NOT NULL,
    failure_type TEXT,                          -- 機械的/電気的/ソフトウェア/操作ミス
    root_cause TEXT,
```

```
    corrective_action TEXT,
    cost INTEGER DEFAULT 0,                    -- 修理費用（円）
    created_at TIMESTAMPTZ DEFAULT now()
);

-- SOP文書
CREATE TABLE mfg_sop_documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    title TEXT NOT NULL,
    process_name TEXT,                          -- 対象工程名
    department TEXT,                            -- 担当部門
    version TEXT NOT NULL DEFAULT '1.0',
    content JSONB NOT NULL,                     -- SOP内容（構造化JSON）
    status TEXT DEFAULT 'draft',                -- draft/review/approved/archived
    approved_by UUID,
    approved_at TIMESTAMPTZ,
    previous_version_id UUID,                  -- 旧版SOP（改訂管理）
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 発注記録（仕入管理）
CREATE TABLE mfg_purchase_orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    po_number TEXT NOT NULL,                   -- 発注番号
    product_code TEXT NOT NULL,                -- 生産指示の製品コード
    supplier_name TEXT NOT NULL,               -- 仕入先名
    order_date DATE NOT NULL,
    required_date DATE NOT NULL,
    payment_terms_days INTEGER DEFAULT 30,     -- 支払条件（日）
    total_amount INTEGER NOT NULL,             -- 発注総額（円）
    status TEXT DEFAULT 'draft',               -- draft/ordered/received/paid
    compliance_warnings JSONB DEFAULT '[]',    -- 下請法チェック結果
    pipeline_result JSONB,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- 発注明細
CREATE TABLE mfg_purchase_order_items (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    purchase_order_id UUID NOT NULL REFERENCES mfg_purchase_orders(id),
    part_code TEXT NOT NULL,
    part_name TEXT NOT NULL,
    quantity DECIMAL(12,2) NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,
    amount INTEGER NOT NULL,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- ISO文書マスタ
CREATE TABLE mfg_iso_documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    document_name TEXT NOT NULL,
    document_type TEXT NOT NULL,                -- manual/procedure/record/form
    iso_clause TEXT,                            -- 対応するISO条項番号
    iso_standard TEXT DEFAULT '9001',           -- 9001/14001/both
    version TEXT NOT NULL DEFAULT '1.0',
    department TEXT,
    last_revised_date DATE,
    expiry_date DATE,
    revision_cycle_years INTEGER DEFAULT 3,
    file_path TEXT,                             -- Supabase Storage パス
```

```
    status TEXT DEFAULT 'active',               -- active/expired/archived
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- ISO監査記録
CREATE TABLE mfg_iso_audit_records (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id UUID NOT NULL REFERENCES companies(id),
    audit_date DATE NOT NULL,
    audit_type TEXT NOT NULL,                  -- internal/external/surveillance
    iso_standard TEXT DEFAULT '9001',
    missing_documents JSONB DEFAULT '[]',
    expiry_alerts JSONB DEFAULT '[]',
    completeness_issues JSONB DEFAULT '[]',
    pipeline_result JSONB,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- RLS（全テーブル必須）
ALTER TABLE mfg_quotes ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_quote_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_production_plans ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_quality_records ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_inventory_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_equipments ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_equipment_failures ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_sop_documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_purchase_orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_purchase_order_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_iso_documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE mfg_iso_audit_records ENABLE ROW LEVEL SECURITY;

-- インデックス
CREATE INDEX idx_mfg_quotes_company ON mfg_quotes(company_id);
CREATE INDEX idx_mfg_quotes_status ON mfg_quotes(company_id, status);
CREATE INDEX idx_mfg_quality_lot ON mfg_quality_records(company_id, lot_number);
CREATE INDEX idx_mfg_inventory_code ON mfg_inventory_items(company_id, item_code);
CREATE INDEX idx_mfg_equipments_type ON mfg_equipments(company_id, equipment_type);
CREATE INDEX idx_mfg_iso_docs_clause ON mfg_iso_documents(company_id, iso_clause);
```

セキュリティ・コンプライアンス

個人情報の取り扱い

製造業BPOにおける個人情報は限定的（従業員情報、得意先担当者情報）だが、
以下のデータは企業秘密として厳格に保護する。

```
保護対象（企業秘密）：
  ・チャージレート（設備別の時間単価）→ 競合に知られると価格交渉で不利
  ・得意先別の利益率テーブル → 取引先に知られると値下げ要求
  ・BOM（部品表）/ 配合レシピ → 製品の核心技術
  ・工程ノウハウ（段取り手順、治具設計）→ 技能のデジタル資産
  ・品質データ（Cp/Cpk、不良率）→ 競合やOEM先への開示リスク
  ・設備投資情報 → 経営判断材料

保護措置：
  ① RLS: 全テーブル company_id ベース。テナント分離必須
  ② 暗号化: チャージレート・利益率テーブルはDB暗号化カラム（Phase 2+でGCP KMS）
  ③ アクセスログ: 見積データ・品質データの閲覧/ダウンロードを全件記録
  ④ エクスポート制限: CSVダウンロードはadminロールのみ
  ⑤ 匿名化: 横断分析時はcompany_id→ハッシュ化。個社特定不可
```

不正競争防止法との関係

条文	内容	対応
第2条第6項	営業秘密の定義（秘密管理性・有用性・非公知性）	チャージレート・BOMを「営業秘密」として管理
第21条	営業秘密侵害罪	従業員退職時のアクセス権即時失効

制約・前提条件

Phase 1の制約

- 図面OCRは2D図面（PDF/画像）のみ。3D CAD対応はPhase 2+
- 生産計画の山崩しは簡易版（前倒し/後ろ倒しのみ）。最適化アルゴリズム（遺伝的アルゴリズム等）はPhase 2+
- SPC計算は簡易3σ法。正式なA2/D3/D4係数テーブルによるXbar-R管理図はPhase 2+
- EOQ計算の発注コスト・保管コストは簡易推定（単価比率）。実績ベースの正確な計算はPhase 2+
- MRP展開は1段階。多段階BOM展開はPhase 2+
- ISO文書のバージョン管理はDB上のみ。Git-likeな差分管理はPhase 2+
- 設備IoTデータの取り込みはPhase 2+。Phase 1は手入力/CSV

ゲノム連携

- チャージレートはゲノムJSON `manufacturing.json` の `charge_rates` セクションに格納
- 材料単価マスタは `manufacturing.json` の `materials` セクションに格納
- 工程マスタは `manufacturing.json` の `processes` セクションに格納
- 得意先別ルール（利益率・納期・品質要求）はゲノムの `customer_rules` セクションに格納
- 新規業種（板金/食品OEM等）はゲノムJSON追加で対応（Type B/D エンジン）

外部連携（Phase 1は直渡し。Phase 2+でAPI連携）

外部システム	連携内容	Phase
kintone	見積管理/受注管理のデータ取得	Phase 1（CSV）
生産管理SaaS（ものレボ/テクノア等）	生産実績/在庫データの同期	Phase 2+
会計ソフト（freee/MF）	仕入データの同期	Phase 2+
IoTゲートウェイ	設備稼働データのリアルタイム取得	Phase 2+